EV Ebene-Vento Technology

FREE COURSE BOOK

HTML & CSS for Beginners

From zero to your first published website

```
<h1>Hello, world!</h1>
h1 { color: #2F5BEA; }
<p>My first website</p>
```

12 lessons | Exercises | Quizzes | Answer key

Written by

Ebene-Vento Technology

Free for all students

HTML & CSS for Beginners

From zero to your first published website

Written by **Ebene-Vento Technology**

A free course book for students of Ebene-Vento Technology. You may share it freely with anyone who wants to learn.

Contact
WhatsApp: +234 706 694 9022
Email: greenhopecover@gmail.com
Website: greenhood1.github.io/Lesson

Copyright © Ebene-Vento Technology. All rights reserved. This book is provided free of charge for learning. Please do not sell it or claim it as your own work.

How to use this book

This book follows the lessons on our website, one chapter for each lesson, but goes deeper with more explanation, more examples and more practice. Use the book and the website together.

Our learning loop

1. **Read** the chapter slowly.
2. **Type** every example in the live editor on the website, or in your code editor. Do not copy and paste.
3. **Change** things and watch what happens. Breaking code on purpose teaches you quickly.
4. **Practise** with the exercises at the end of the chapter.
5. **Test yourself** with the quiz. Answers are at the back of the book.
6. **Ask** your teacher when you are stuck. Live help is available.

What you will build

By the last chapter you will have built and published a real website that works on phones and computers. You can use it for yourself, a small business or a group.

Certificate

When you finish all lessons, pass every quiz and publish your final project, you can request a verified certificate of completion on our website.

Symbols used in this book

> **Tip** A shortcut or good habit that will save you time.

> **Watch out** A common mistake that catches many beginners.

Contents

Introduction: Welcome to HTML and CSS 4

Lesson 1: Your First Web Page 7

Lesson 2: Links and Lists 9

Lesson 3: Images and Tables 11

Lesson 4: Forms 13

Lesson 5: Styling with CSS 15

Lesson 6: Classes and IDs 17

Lesson 7: Layout with Flexbox 19

Lesson 8: Layout with Grid 21

Lesson 9: Responsive Design for Phones 23

Lesson 10: Final Project: Build Your Own Website 25

Lesson 11: Publish Your Website 28

Quizzes and answer key 30

Cheat sheet 33

Glossary 35

What next? 36

INTRODUCTION

Welcome to HTML and CSS

In this chapter you will learn

- What HTML and CSS are and how they work together
- How a website reaches your screen
- The tools you need to start
- The anatomy of a tag and of a CSS rule
- The skeleton every web page shares

What are HTML and CSS?

Every website you have ever visited, from a small shop page to a giant social network, is built from three languages: HTML, CSS and JavaScript. This book teaches the first two, because they are the foundation. Once you know them, everything else on the web becomes easier to learn.

HTML (HyperText Markup Language) describes the structure and content of a page: headings, paragraphs, images, links, lists and forms. Think of a house. HTML is the walls, rooms, doors and windows.

CSS (Cascading Style Sheets) describes how the page looks: colors, fonts, spacing and layout. In the house comparison, CSS is the paint, the furniture and the decoration. The same HTML can look completely different with different CSS.

How a website reaches your screen

When you type an address such as `greenhood1.github.io/Lesson` into your browser, your phone asks a computer called a **server** for the page. The server sends back files: an HTML file, usually a CSS file, and any images. Your browser reads the HTML to build the structure, then applies the CSS to style it. All of this happens in a fraction of a second.

A **browser** (Chrome, Firefox, Safari, Edge) is the program that reads these files and shows you the result. Because every browser follows the same rules, your page looks almost the same for everyone.

Tools you need

- **A browser.** Chrome is a good choice. You already have one.
- **A code editor.** This is where you type your code. On a computer, use Visual Studio Code (free). On Android, try Acode. On iPhone, try Textastic. Any plain-text editor works.
- **The live editor on our website.** It needs nothing installed. You type code on one side and see the result instantly on the other.
- **A folder** on your device where you keep your website files together.

> **Tip** Do not use a word processor like Microsoft Word to write code. It adds hidden formatting that breaks web pages. Use a code editor or the live editor.

The anatomy of an HTML element

HTML is made of **elements**. An element usually has an opening tag, some content and a closing tag. The closing tag has a slash.

```
<p class="intro">Hello, world!</p>

<p ...>          opening tag
class="intro"    an attribute (extra information)
Hello, world!    the content
</p>              closing tag
```

An **attribute** gives extra information about an element. It sits inside the opening tag and has a name and a value in quotes, such as `class="intro"` or `href="https://example.com"`. Some elements, such as `<img>` and `<br>`, have no content and no closing tag.

The skeleton of every web page

Every HTML page follows the same basic layout. Learn this by heart, because you will use it in every project.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>My first page</title>
</head>
<body>
  <h1>Welcome</h1>
  <p>This is visible content.</p>
</body>
</html>
```

- `<!DOCTYPE html>` tells the browser this is a modern HTML page.
- `<html lang="en">` wraps the whole page and states the language.
- `<head>` holds information about the page that visitors do not see directly.
- `<meta charset="UTF-8">` lets the page show symbols such as the naira sign correctly.
- `<meta name="viewport" ...>` makes the page fit phone screens (Lesson 9 explains it).
- `<title>` is the text shown on the browser tab.
- `<body>` holds everything visitors see.

The anatomy of a CSS rule

```
h1 {
  color: blue;
  font-size: 32px;
}

h1            selector (what to style)
color         property (what to change)
blue          value (what to change it to)
color: blue;  one declaration
```

There are three places to write CSS. **Inline** CSS goes inside one tag with the `style` attribute. **Internal** CSS goes inside a `<style>` tag in the head. **External** CSS lives in its own `.css` file linked with `<link rel="stylesheet" href="style.css">`. Beginners usually start with the internal style tag; real websites use external files.

Good habits from day one

- Save HTML files with the `.html` ending, in lowercase, with no spaces: `about-us.html`.
- Indent your code so you can read it. Two spaces per level is common.
- Close every tag you open.
- Make one small change at a time and check the result.
- Type the examples yourself instead of copying them. Your fingers learn too.
- Read error-looking results calmly. A broken page almost always has a missing quote, bracket or closing tag.

Practice exercises

1. Open the live editor on the website and press Try it on the introduction lesson. Change the title and heading text.
2. Change the heading color from blue to green, then to any color you like.
3. Draw the skeleton of a page from memory on paper, then compare it with the one in this chapter.

LESSON 1

Your First Web Page

In this chapter you will learn

- Headings, paragraphs and line breaks
- Bold and italic text
- Comments and special characters
- How browsers treat spaces and nesting

Headings

HTML gives you six heading levels, from `<h1>` (the biggest and most important) to `<h6>` (the smallest). Use one `<h1>` per page for the main title, then `<h2>` for sections and `<h3>` for sub-sections. Choose headings by meaning, not by size. You can change size later with CSS.

```
<h1>My Website</h1>
<h2>About me</h2>
<h3>My hobbies</h3>
```

Paragraphs and line breaks

Use `<p>` for every paragraph. Browsers add space above and below each one. To break a line inside a paragraph, use `<br>`. To draw a horizontal line between sections, use `<hr>`. Both have no closing tag.

```
<p>Welcome to my page.<br>I am learning HTML.</p>
<hr>
<p>This is a second paragraph.</p>
```

Bold, italic and emphasis

`<b>` makes text bold and `<i>` makes it italic. Their meaningful cousins are `<strong>` (important text) and `<em>` (stressed text). Browsers show them the same way, but screen readers and search engines understand the meaning, so prefer `<strong>` and `<em>` when the text really is important.

```
<p>This is <b>bold</b>, this is <i>italic</i>,
and this is <strong>important</strong>.</p>
```

Comments

A comment is a note for you that the browser ignores. It starts with `<!--` and ends with `-->`. Use comments to label sections of a long page or to temporarily switch off some code.

```
<!-- Main heading starts here -->
<h1>Hello</h1>
<!-- <p>This paragraph is hidden for now</p> -->
```

How browsers treat spaces

Browsers collapse extra spaces and line breaks into a single space. Pressing Enter five times in your code does not create five blank lines on the page. To get space, use `<br>`, separate paragraphs, or CSS spacing (Lesson 5).

Special characters

Some characters have a special meaning in HTML. To show them as text, write them as **entities**: `<` shows <, `>` shows >, `&` shows &, and `©` shows the copyright sign.

Nesting

Elements can sit inside each other. This is called **nesting**. The rule is simple: the tag you open last must be the first one you close.

```
<p>This is <strong>very <em>important</em></strong> news.</p>

Wrong:  <strong><em>text</strong></em>
Right:  <strong><em>text</em></strong>
```

> **Watch out** Common mistakes: forgetting a closing tag, mixing up the order of closing tags, and using headings just to make text big.

Practice exercises

1. Create a page with your name as `<h1>`, a short introduction in a paragraph, and one word in bold.
2. Add an `<h2>` called My Favourites and two paragraphs below it, separated by `<hr>`.
3. Add a comment above each section. Then comment out one paragraph and check that it disappears.

LESSON 2

Links and Lists

In this chapter you will learn

- Making links to other pages and websites
- Email, phone and in-page links
- Bulleted, numbered and nested lists

Links

Links are what make the web a web. A link uses the anchor tag `<a>`. The `href` attribute holds the destination, and the text between the tags is what the visitor clicks.

```
<a href="https://example.com">Visit Example</a>
```

Absolute and relative links

An **absolute** link has the full address, starting with `https://`. Use it for other websites. A **relative** link points to a file in your own project, such as `about.html` or `pages/contact.html`. Relative links keep working when you move your site to a new host.

```
<a href="https://wikipedia.org">Wikipedia</a>   absolute
<a href="about.html">About us</a>                relative
```

Opening in a new tab

Add `target="_blank"` to open a link in a new tab. For safety, also add `rel="noopener"`.

```
<a href="https://example.com" target="_blank" rel="noopener">Open in new tab</a>
```

Email, phone and WhatsApp links

```
<a href="mailto:you@example.com">Email me</a>
<a href="tel:+2347000000000">Call me</a>
<a href="https://wa.me/2347000000000">Chat on WhatsApp</a>
```

These are very useful for local businesses: one tap and the visitor is talking to you.

Links inside a page

Give an element an `id`, then link to it with a `#`. This is how the menu on a one-page site scrolls to each section.

```
<a href="#contact">Go to contact</a>
...
<h2 id="contact">Contact</h2>
```

Lists

Use `<ul>` for a bulleted (unordered) list and `<ol>` for a numbered (ordered) list. Every item goes in an `<li>`. Lists can be nested by placing a list inside an `<li>`.

```
<ul>
  <li>Rice</li>
  <li>Beans
    <ul>
      <li>Brown beans</li>
      <li>White beans</li>
    </ul>
  </li>
</ul>

<ol>
  <li>Open the editor</li>
  <li>Write code</li>
</ol>
```

> **Tip** Use a list whenever you have several items of the same kind. Menus, steps and features are all lists.

Practice exercises

1. Make a bulleted list of five things you want to buy and a numbered list of the steps to make your favourite meal.
2. Add a link to a website you like that opens in a new tab.
3. Add a WhatsApp link with your own number and test it on your phone.

LESSON 3

Images and Tables

In this chapter you will learn

- Adding images the right way
- Choosing image formats and sizes
- Building tables with rows, headers and cells

Images

The `<img>` tag shows a picture. It has no closing tag and two important attributes: `src` (where the file is) and `alt` (a short description).

```
<img src="photo.jpg" alt="A student writing code on a laptop">
```

The `alt` text matters for three reasons. Screen readers read it to blind visitors. It appears if the image fails to load, which happens often on slow networks. Search engines use it to understand the picture.

Controlling size

You can set `width` and `height` in pixels. Set only the width and the browser keeps the proportions. A very common CSS rule makes images fit any screen:

```
img {
  max-width: 100%;
  height: auto;
}
```

Choosing a format

- **JPG** is best for photographs. Small files, no transparency.
- **PNG** is best for logos, screenshots and images that need a transparent background.
- **SVG** is best for icons and simple graphics. It stays sharp at any size.
- **WebP** is a modern format that is smaller than JPG and PNG and works in all current browsers.

> **Tip** Data is expensive. Before using a photo, shrink it. A 5 MB picture straight from a phone camera will make your page slow. Aim for under 300 KB per image.

Where is my file?

If `src="photo.jpg"` shows a broken image, the file is not where the browser looks. Check three things: the file is in the same folder as your HTML file, the name matches exactly (including capital letters and the ending), and there are no spaces in the name.

Tables

Tables show data in rows and columns. `<table>` wraps everything. `<tr>` is a row. `<th>` is a header cell and `<td>` is a data cell.

```
<table border="1" cellpadding="8">
  <tr>
    <th>Item</th>
    <th>Price</th>
  </tr>
  <tr>
    <td>Logo design</td>
    <td>₦10,000</td>
  </tr>
  <tr>
    <td>Flyer</td>
    <td>₦5,000</td>
  </tr>
</table>
```

For bigger tables, group rows with `<thead>` (headings) and `<tbody>` (data). To make a cell span several columns, use `colspan="2"`; to span rows, use `rowspan`.

> **Watch out** Do not use tables to lay out a whole page. Tables are for data. For page layout use Flexbox and Grid (Lessons 7 and 8).

Practice exercises

1. Add a picture to your page. Write a useful alt text for it.
2. Build a table with three columns (Name, Subject, Score) and four rows of data.
3. Add `max-width: 100%; height: auto;` for images in a style tag and check the result.

LESSON 4

Forms

In this chapter you will learn

- Collecting information with form fields
- Labels, input types and validation
- What happens to the data after the form is filled

The form tag

A form groups fields that a visitor fills in. Inside `<form>` you place inputs, labels, text areas and a button.

Inputs and types

The `<input>` tag makes a field. Its `type` attribute decides what kind: `text`, `email`, `password`, `number`, `date`, `tel` and more. Choosing the right type gives phone users the right keyboard and checks the value.

```
<input type="text" placeholder="Your name">
<input type="email" placeholder="you@example.com">
<input type="password">
<input type="number" min="1" max="10">
```

Labels

Every field needs a `<label>`. A label tells the visitor what to type, and tapping the label focuses the field. Connect them with matching `for` and `id` values, or wrap the input inside the label.

```
<label for="name">Full name</label>
<input type="text" id="name" name="name">
```

Text areas, menus, checkboxes and radio buttons

```
<textarea rows="4"></textarea>

<select>
  <option>Beginner</option>
  <option>Intermediate</option>
</select>

<input type="checkbox" id="news">
<label for="news">Send me updates</label>

<input type="radio" name="plan" value="free"> Free
<input type="radio" name="plan" value="paid"> Paid
```

Radio buttons with the same `name` form a group: only one can be chosen.

Buttons and simple validation

```
<button type="submit">Send</button>
<input type="email" required>
```

The `required` attribute stops the form from being sent while the field is empty. Browsers add checks for email and number types automatically.

What happens to the data?

HTML only builds the form. To receive what people type, something must process it: a server program (for example PHP), a form service, or JavaScript. A simple trick for a small business is to turn the form into a WhatsApp message with JavaScript, which is exactly how the free-spot form on our website works. You will meet JavaScript in your next course.

> **Watch out** Never ask for sensitive information such as card numbers or passwords unless you know how to protect it. Learn about security first.

Practice exercises

1. Build a registration form with name, email, phone, a course choice (select) and a message box.
2. Make the name and email fields required. Try submitting with empty fields.
3. Add a checkbox with a label that you can tap to toggle.

LESSON 5

Styling with CSS

In this chapter you will learn

- Writing CSS rules and choosing colors
- Fonts, sizes and text alignment
- The box model: content, padding, border, margin
- Borders, rounded corners and shadows

Selecting elements

The simplest selector is an element name. A rule for `p` changes every paragraph on the page.

```
body {
  background: #f4f7fb;
  font-family: Arial, sans-serif;
}
h1 { color: #14213d; }
p  { font-size: 18px; line-height: 1.6; }
```

Colors

You can write colors three ways. **Names** such as `tomato` or `navy`. **Hex codes** such as `#2F5BEA` (a hash and six characters). **RGB** values such as `rgb(47, 91, 234)`. Hex codes are the most common. Search for a color picker online to find any color.

Fonts and text

- `font-family` chooses the typeface. Always add a fallback: `font-family: Georgia, serif;`
- `font-size` sets the size: `20px`, or relative units like `1.2rem`.
- `font-weight: bold;` for thick text.
- `text-align: center;` aligns text left, right, center or justify.
- `line-height: 1.6;` adds comfortable space between lines.
- `text-decoration: none;` removes the underline from links.

Units

`px` is a fixed pixel size. `%` is relative to the parent. `rem` is relative to the page's base font size, usually 16px, so `1.5rem` is 24px. `vw` and `vh` are relative to the screen width and height. Prefer `rem` and `%` for flexible pages.

The box model

Every element is a rectangular box with four layers. From inside out: the **content**, the **padding** (space between content and border), the **border**, and the **margin** (space outside the border, between this box and its neighbours).

```
+------------------ margin ------------------+
|  +--------------- border --------------+  |
|  |  +------------ padding -----------+  |  |
|  |  |         content goes here      |  |  |
|  |  +--------------------------------+  |  |
|  +--------------------------------------+  |
+--------------------------------------------+
```

```
p {
  padding: 12px;
  border: 2px solid tomato;
  border-radius: 8px;
  margin: 20px 0;
}
```

> **Tip** Add `box-sizing: border-box;` to everything with `* { box-sizing: border-box; }`. It makes widths include padding and border, which is far easier to calculate.

Shadows and rounded corners

```
.card {
  border-radius: 12px;
  box-shadow: 0 4px 12px rgba(0,0,0,0.15);
}
```

Block and inline

Block elements such as `<p>` and `<h1>` start on a new line and take the full width. Inline elements such as `<a>` and `<strong>` stay inside the line. `display: inline-block` gives an inline element a width and height.

Practice exercises

1. Style a page with a background color, a custom font, and a heading in your favourite color.
2. Give a paragraph padding, a border and rounded corners. Change each number and watch what moves.
3. Create a button-like link with padding, a background color, white text and no underline.

LESSON 6

Classes and IDs

In this chapter you will learn

- Styling some elements and not others
- The difference between a class and an id
- Grouping, nesting and hover effects
- How the browser decides which rule wins

Why classes?

Styling every `<p>` the same is limiting. A **class** lets you name a group of elements and style only that group. Add `class="note"` to the tag, and write `.note` in CSS. The dot means class.

```
<p class="note">Remember this.</p>
<p>A normal paragraph.</p>

.note {
  background: #ffd23f;
  padding: 10px;
  border-radius: 6px;
}
```

Many elements can share a class, and one element can have several classes separated by spaces: `class="note big"`.

IDs

An **id** names one unique element on the page. In CSS, use a hash: `#title`. An id must be used only once per page. Ids are also used for in-page links.

```
<h2 id="title">I am unique</h2>

#title { color: #2f5bea; }
```

> **Tip** Use classes for styling and keep ids for links and special cases. Classes are far more flexible.

Grouping and descendant selectors

```
h1, h2, h3 {              /* grouping: three selectors, one rule */
  font-family: Georgia, serif;
}

.card p {                 /* only paragraphs inside .card */
  color: #51607a;
}
```

Hover and other states

A **pseudo-class** styles an element in a special state. `:hover` applies when the mouse is over it (on phones, when tapped). Add `transition` to make the change smooth.

```
a.button {
  background: #2f5bea;
  color: white;
  padding: 10px 18px;
  border-radius: 8px;
  text-decoration: none;
  transition: background 0.2s;
}
a.button:hover { background: #1d3fb0; }
```

Which rule wins?

When two rules conflict, the browser follows **specificity**: an id beats a class, a class beats an element, and when they are equal, the rule written later wins. Inline `style` beats them all. If a style will not apply, check whether a more specific rule is overriding it.

Naming classes

Choose names that describe purpose, not appearance: `.warning` is better than `.red-box`. Use lowercase and hyphens, no spaces: `.main-menu`.

Practice exercises

1. Create a class called `warning` with a red background and white text. Use it on two paragraphs.
2. Create a `.button` class and make it change color on hover.
3. Give your page a card: a `div` with a class, padding, a border and a shadow, containing a heading and paragraph.

LESSON 7

Layout with Flexbox

In this chapter you will learn

- Placing items in a row or column
- Spacing and aligning items
- Building a navigation bar
- Centering anything

The idea

Flexbox is a layout system for arranging items along one direction, either a row or a column. You turn it on for the **container**, and its direct children become **flex items**.

```
<div class="row">
  <div class="box">One</div>
  <div class="box">Two</div>
  <div class="box">Three</div>
</div>

.row {
  display: flex;
  gap: 12px;
}
```

Main axis and cross axis

In a row, the main axis runs left to right and the cross axis runs top to bottom. With `flex-direction: column`, they swap. `justify-content` aligns along the main axis; `align-items` aligns along the cross axis.

Spacing with justify-content

- `flex-start` (default): items packed at the start
- `center`: items grouped in the middle
- `flex-end`: items packed at the end
- `space-between`: first and last at the edges, equal gaps between
- `space-around` and `space-evenly`: equal space around items

Wrapping and growing

By default items try to stay on one line. Add `flex-wrap: wrap;` so they drop onto the next line when there is no room. Give an item `flex: 1;` so it grows to share the free space equally with others.

Example: a navigation bar

```
<nav class="menu">
  <a href="#">Home</a>
  <a href="#">About</a>
  <a href="#">Contact</a>
</nav>

.menu {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 12px 20px;
  background: #14213d;
}
.menu a { color: white; text-decoration: none; }
```

Centering anything

Centering used to be hard. With Flexbox it takes three lines, and it works on any container with a height.

```
.center {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 300px;
}
```

Practice exercises

1. Build three boxes in a row. Try each value of `justify-content` and describe the difference.
2. Change `flex-direction` to `column`. What happens to the gap?
3. Build a navigation bar with a logo on the left and three links on the right.

LESSON 8

Layout with Grid

In this chapter you will learn

- Creating rows and columns
- Using fr units, repeat and gap
- Making items span several cells
- Choosing between Flexbox and Grid

The idea

Grid is a two-dimensional layout system. Where Flexbox arranges items in one line, Grid arranges them in rows and columns at once. It is excellent for galleries, dashboards and whole page layouts.

```
<div class="grid">
  <div class="box">A</div>
  <div class="box">B</div>
  <div class="box">C</div>
  <div class="box">D</div>
</div>

.grid {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 12px;
}
```

The fr unit and repeat

`1fr` means one fraction of the free space. Three `1fr` columns are equal. `repeat(3, 1fr)` is a short way to write the same thing. Mix units too: `200px 1fr` gives a fixed sidebar and a flexible main area.

Spanning cells

```
.wide {
  grid-column: span 2;   /* take two columns */
}
.tall {
  grid-row: span 2;      /* take two rows */
}
```

A gallery that adapts by itself

This one line creates as many columns of at least 160px as will fit. On a phone you get one or two columns; on a laptop you get many. No media query needed.

```
.gallery {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(160px, 1fr));
  gap: 12px;
}
```

Flexbox or Grid?

Use Flexbox for	Use Grid for
A row of buttons or a menu	A photo gallery
Centering one item	A full page layout with header, sidebar, main and footer
Items in one direction	Rows and columns at the same time

Practice exercises

1. Build a grid of six boxes with three columns. Change it to two columns.
2. Make the first box span two columns.
3. Create a gallery with `auto-fit` and shrink the Result panel to see it adapt.

LESSON 9

Responsive Design for Phones

In this chapter you will learn

- Why most of your visitors are on phones
- The viewport meta tag
- Flexible sizes and media queries
- Testing on different screens

Why responsive?

In Nigeria and across Africa, most people browse on phones. A page that only looks right on a laptop loses most of its visitors. **Responsive design** means one page that adapts to any screen width.

The viewport meta tag

Without it, phones pretend to be wide desktops and shrink your page into tiny text. This line in the head is essential:

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

Flexible building blocks

- Use `%`, `rem` and `fr` units instead of fixed pixel widths.
- Use `max-width` instead of `width` for containers: `max-width: 900px; margin: 0 auto;`
- Make images flexible with `max-width: 100%; height: auto;`
- Let items wrap with `flex-wrap: wrap` or use `auto-fit` grids.

Media queries

A **media query** applies CSS only when a condition is true, usually about the screen width. **Mobile-first** means you write the phone layout first, then add queries for bigger screens with `min-width`.

```
.cards {
  display: flex;
  flex-direction: column;   /* phone: stacked */
  gap: 12px;
}

@media (min-width: 600px) {
  .cards {
    flex-direction: row;    /* wider screens: side by side */
  }
  .card { flex: 1; }
}
```

Common breakpoints

Width	Typical device
up to 480px	Small phones
600px and above	Large phones and small tablets
900px and above	Tablets and small laptops
1200px and above	Desktops

> **Tip** Do not design for exact devices. Resize your window or your Result panel and add a breakpoint wherever the layout starts to look awkward.

Testing

Open your page on a real phone. On a computer, open the browser's developer tools and use the device toolbar to simulate phone sizes. Also test in landscape mode.

Practice exercises

1. Make a page with three cards that are stacked on phones and in a row on screens 600px or wider.
2. Make the heading smaller on phones and larger on wide screens using a media query.
3. Add the viewport meta tag, then test the page on your phone with and without it. Describe the difference.

LESSON 10

Final Project: Build Your Own Website

In this chapter you will learn

- Planning a small website
- Putting every lesson together
- A checklist for quality

Choose your project

Build a one-page website about something real: yourself, a small business you want to start, a church or school group, or a service you offer. Real content keeps you motivated and gives you a site you can actually use.

Step 1: Plan

- **Purpose:** what should a visitor do after reading? Call you, message you, buy?
- **Sections:** header, about, services or products, contact, footer.
- **Content:** write the real text before you code. Gather two or three small images.
- **Colors and fonts:** pick two main colors and one or two fonts.

Step 2: Build the structure with HTML

Start with the skeleton. Use `<header>`, `<section>` and `<footer>` to group content. They are meaningful tags that help search engines and screen readers.

Step 3: Style with CSS

Style the whole page first (body, fonts, colors), then each section, then details such as buttons and hover effects.

Starter project

```
<header>
  <h1>Amaka Designs</h1>
  <p>Affordable logos and flyers in Lagos</p>
</header>

<section>
  <h2>About me</h2>
  <p>I help small businesses look professional.</p>
</section>

<section>
  <h2>Services</h2>
  <ul>
    <li>Logo design</li>
    <li>Flyers and posters</li>
    <li>Social media graphics</li>
  </ul>
</section>

<footer>Contact: 0700 000 0000</footer>
```

```
body { margin: 0; font-family: sans-serif; color: #14213d; }
header { background: #14213d; color: #fff; padding: 28px 20px; }
header p { color: #ffd23f; margin: 0; }
section { padding: 16px 20px; }
h2 { color: #2f5bea; }
footer { background: #2f5bea; color: #fff; padding: 14px 20px; }
```

Step 4: Make it yours

- Replace the name, text and services with your own.
- Add your logo or a photo with a good alt text.
- Add a contact link: WhatsApp, phone and email.
- Add a services section using a grid of cards.
- Make sure it looks good on a phone.

Quality checklist

- Every tag is closed and the page has the full skeleton.
- The page title is meaningful.
- Every image has an alt text and is under 300 KB.
- Text is easy to read: enough size and contrast between text and background.
- Links work and the contact details are correct.
- The page looks good on a phone and a computer.
- You have read the whole page aloud once and fixed typos.

Practice exercises

1. Complete your project using at least: headings, a list, an image, a link, a class, Flexbox or Grid, and one media query.
2. Show the page to a friend and ask them what they would do next. If it is not what you wanted, improve the design.
3. Prepare your project to publish in the next lesson: rename the main file to `index.html`.

LESSON 11

Publish Your Website

In this chapter you will learn

- How hosting and domains work
- Publishing for free with GitHub Pages and Netlify
- Updating your site
- Where to go next

How publishing works

Your website is just files. To let the world see it, you put the files on a **host**: a computer that is always on and connected to the internet. The host gives you an address. You can get a free address from a free host, or you can buy your own name such as `yourname.com`, called a **domain**.

Before you publish

- Name your main page `index.html`. Hosts show this file when someone visits your address.
- Use lowercase file names without spaces. Hosting is usually case-sensitive: `Photo.JPG` and `photo.jpg` are different files.
- Keep images in the same folder or an `images` folder, and use relative paths.
- Test the page offline first.

Option 1: GitHub Pages (free)

- Create a free account at github.com.
- Tap New repository, give it a name, set it to Public and create it.
- Choose Add file, then Upload files, and add `index.html` and your images. Commit the changes.
- Open Settings, then Pages. Under Source choose Deploy from a branch, select the `main` branch and the root folder, and save.
- Wait one or two minutes. Your link appears on that screen, like `yourname.github.io/repository-name`.

Option 2: Netlify Drop (free, fastest)

- Go to `app.netlify.com/drop`.
- Drag your project folder (or your `index.html`) onto the page.
- In under a minute you get a live link you can share.

Updating your site

On GitHub, open the file, tap the pencil to edit, make your changes and commit. On Netlify, drag the updated folder again. After changes, if you still see the old page, clear your cache or open the link in a private tab.

Using your own domain

A custom domain is optional and costs a small yearly fee from a domain registrar. You connect it in your host's settings by following their steps. Start with the free address; buy a domain when you have something worth sharing.

Common problems

Problem	Likely cause and fix
404 page not found	The main file is not named `index.html`, or the site is still building. Wait two minutes and check the name.
Images do not show	Wrong file name, wrong capital letters, or the image was not uploaded. Check the path.
Old version still showing	Browser cache. Hard refresh or open a private tab.
Page looks tiny on a phone	The viewport meta tag is missing.

Where to go next

- **JavaScript:** makes pages interactive, with menus, calculators, quizzes and games.
- **Git and GitHub:** save versions of your work properly.
- **Frameworks and tools:** learn after you are comfortable with the basics.
- **PHP and databases:** build sites that store data, such as sign-ups and logins.
- **Real projects:** build a site for a local business. Practice with real clients is the fastest way to grow.

Practice exercises

1. Publish your final project and open the link on a phone that is not yours.
2. Change one line of text and update the live site.
3. Send your link to Ebene-Vento Technology on WhatsApp to request your certificate review.

APPENDIX A

Quizzes and answer key

Try each question first, without looking. The correct answer is shown after each question.

Introduction

1. What does CSS control?

a) The structure of a page b) The colors, fonts and layout c) The page address

Answer: b) The colors, fonts and layout

2. Which tag holds everything visitors see?

a) <head> b) <body> c) <title>

Answer: b) <body>

Lesson 1: Your first web page

1. Which heading is the largest?

a) <h6> b) <h1> c) <h3>

Answer: b) <h1>

2. Which tag makes a paragraph?

a) <p> b)
 c) <hr>

Answer: a) <p>

Lesson 2: Links and lists

1. Which attribute holds a link's address?

a) src b) href c) alt

Answer: b) href

2. Which tag makes a numbered list?

a) b) c)

Answer: b)

Lesson 3: Images and tables

1. What does the alt attribute do?

a) Sets the image size b) Describes the image c) Adds a border

Answer: b) Describes the image

2. Which tag creates a table row?

a) <td> b) <tr> c) <th>

Answer: b) <tr>

Lesson 4: Forms

1. Which input type is meant for email addresses?

a) text b) email c) number

Answer: b) email

2. Which tag names a form field?

a) <label> b) <form> c) <option>

Answer: a) <label>

Lesson 5: Styling with CSS

1. Which property adds space inside an element?

a) margin b) padding c) gap

Answer: b) padding

2. Which is a valid hex color?

a) #2F5BEA b) 2F5BEA# c) hex(2F5BEA)

Answer: a) #2F5BEA

Lesson 6: Classes and IDs

1. How do you select a class in CSS?

a) .name b) #name c) *name

Answer: a) .name

2. How many times should one id appear on a page?

a) Once b) Many times c) Never

Answer: a) Once

Lesson 7: Layout with Flexbox

1. Which line makes a flex container?

a) display: flex; b) flex: container; c) layout: row;

Answer: a) display: flex;

2. What does gap do?

a) Adds space between items b) Changes the font c) Adds a border

Answer: a) Adds space between items

Lesson 8: Layout with Grid

1. What does repeat(3, 1fr) create?

a) Three equal columns b) Three rows only c) Three colors

Answer: a) Three equal columns

2. Which property turns on grid?

a) display: grid; b) grid: on; c) layout: grid;

Answer: a) display: grid;

Lesson 9: Responsive design for phones

1. What is a media query used for?

a) Playing videos b) Changing styles at certain screen sizes c) Adding images

Answer: b) Changing styles at certain screen sizes

2. Why design for phones first?

a) Most visitors use phones b) Phones cannot read CSS c) It is required by law

Answer: a) Most visitors use phones

Lesson 10: Final project, build your own website

1. Which tag groups the main areas of a page?

a) <section> b)
 c) <hr>

Answer: a) <section>

2. Where can small pages keep their CSS?

a) Inside a <style> tag b) Inside <title> c) Inside <a>

Answer: a) Inside a <style> tag

Lesson 11: Publish your website

1. What should your main file be called?

a) home.txt b) index.html c) site.css

Answer: b) index.html

2. Can you host a public site on GitHub Pages for free?

a) Yes b) No, always paid c) Only on weekends

Answer: a) Yes

APPENDIX B

Cheat sheet

Common HTML tags

Tag	Purpose
<h1> to <h6>	Headings
<p>	Paragraph
<a href>	Link
<img src alt>	Image
	Lists and list items
<table> <tr> <th> <td>	Tables
<form> <input> <label>	Forms and fields
<button>	Button
	Generic container
<header> <section> <footer>	Page areas
  <hr>	Line break and line
	Important and stressed text

Common CSS properties

Property	What it does
color	Text color
background	Background color or image
font-family / font-size	Typeface and size
text-align	Align text
padding / margin	Space inside / outside
border / border-radius	Outline and rounded corners
width / max-width	Size of an element
display: flex / grid	Turn on a layout system
justify-content / align-items	Align flex items
gap	Space between flex or grid items
grid-template-columns	Define grid columns
@media (min-width: ...)	Styles for bigger screens

APPENDIX C

Glossary

Term	Meaning
Attribute	Extra information inside an opening tag, such as href or class.
Browser	The program that reads web files and shows pages, such as Chrome.
Class	A name given to elements so CSS can style them as a group.
Comment	A note in code that the browser ignores.
CSS	Cascading Style Sheets, the language that styles web pages.
Domain	A web address you own, such as yourname.com.
Element	A complete piece of HTML: opening tag, content and closing tag.
Flexbox	A CSS system for arranging items in one row or column.
Grid	A CSS system for arranging items in rows and columns.
Host	A computer that stores your website files and serves them to visitors.
HTML	HyperText Markup Language, the language that structures web pages.
Id	A unique name for one element on a page.
Margin	Space outside an element's border.
Media query	CSS that applies only at certain screen sizes.
Padding	Space inside an element, between content and border.
Property	The thing CSS changes, such as color or font-size.
Responsive	A design that adapts to any screen size.
Selector	The part of a CSS rule that says what to style.
Server	A computer that sends web pages to browsers.
Tag	A keyword in angle brackets that marks an HTML element.
Viewport	The visible area of a page on a device screen.

APPENDIX D

What next?

Congratulations on reaching the end. You now know how websites are built and published. Keep practising: build one small page every week and your skills will grow quickly.

Continue with Ebene-Vento Technology

- **1-on-1 live sessions:** personal help from a teacher on your own project.
- **Verified certificate:** proof of completion with a unique ID that anyone can check on our website.
- **Websites and web apps:** we also build websites and web apps for small businesses.
- **Next courses:** JavaScript, PHP and MySQL.

Get in touch

WhatsApp: +234 706 694 9022
Email: greenhopecover@gmail.com
Website: greenhood1.github.io/Lesson

Thank you for learning with us.

Ebene-Vento Technology